

Deep Q-learning Approach for Congestion Problem In Smart Cities

Nada FAQIR
LISAC Laboratory
Sidi Mohamed Ben Abdellah
University Fez, Morocco
nada.faqir@usmba.ac.ma

Noureddine EN-NAHNAHI
LISAC Laboratory
Sidi Mohamed Ben Abdellah
University Fez, Morocco
noureddine.en-nahnahi@usmba.ac.ma

Jaouad BOUMHIDI
LISAC Laboratory
Sidi Mohamed Ben Abdellah
University Fez, Morocco
jaouad.boumhidi@usmba.ac.ma

Abstract— Traffic congestion is a critical problem in urban area. In this study, our objective is the control of traffic lights in an urban environment, in order to avoid traffic jams and optimize vehicle traffic; we aim to minimize the total waiting time. Our system is based on a new paradigm, which is deep reinforcement learning; it can automatically learn all the useful characteristics of traffic data and develop a strategy optimizing adaptive traffic light control. Our system is coupled to a microscopic simulator based on agents (Simulation of Urban MObility - SUMO) providing a synthetic but realistic environment in which the exploration of the results of potential regulatory actions can be carried out.

Keywords—Traffic control, Adaptive systems, traffic optimization, urban mobility.

I. INTRODUCTION

The evolution of the dynamics of transit in cities means that the organization of traffic has quickly become a problem, hence the introduction of traffic lights, signs and public transport planning. The peculiarity of car traffic is that road infrastructure designed according to a demand projected at a certain point in time to meet a collective optimum while each individual seeks to optimize his or her travel while satisfying criteria placed at the individual level. This makes car traffic a phenomenon that is difficult to analyze and optimize, and although simulation models are essential tools for its analysis and understanding, it has a number of characteristics that also make it difficult to model and generate traffic flow problems that can lead to congestion and queuing with corresponding delays. The phenomenon of road traffic congestion is thus a crucial socioeconomic problem that requires the search for appropriate solutions that can be implemented in practice. At the same time, infrastructure enlargement programs are expensive and technically difficult to implement, so optimizing the use of existing infrastructure, both in terms of capacity and safety, is therefore a major economic and social challenge. It is therefore only possible through a better understanding of the mechanisms governing all traffic phenomena. It has thus become essential to focus on dynamic traffic regulation systems, the aim of which is to operate existing infrastructures more efficiently in real-time, by the use of proven techniques. Our work focuses on defining the traffic light control problem as a reinforcement learning problem by modeling it as a Markov decision process [1]. Regarding the regulation of traffic flows associated with advances in machine learning, it is possible to use virtual environments as tools to explore the results of possible regulatory actions in specific situations, within a framework

of reinforcement learning [2]. In this study, our intersection simulation was carried out in an urban mobility simulation tool (SUMO) [2] which makes an Application Programming Interface (API) available for coupling with external programs, the overall traffic flow is described with exploiting the statistics collected by SUMO. With this method, a significant reduction in waiting times is obtained. The close relationship that exists between vehicle waiting times and vehicle delay leads us to say that reducing the waiting time of a vehicle will also decrease its delay.

II. BACKGROUND

A. Reinforcement learning

Reinforcement learning (RL) [1] is a goal-oriented machine learning algorithm. In a reinforcement learning problem, where the agent interacts with the environment (the intersection) at time steps $t = 0, 1, 2, \dots$ with the aim to reduce waiting times of all vehicles in the long run, and thus alleviate congestion. This agent needs a detailed description (state S_t) of this environment at the beginning of the time step t in order to make a decision (action A_t) concerning traffic lights.

Once the action is taken, the vehicles start moving and the environment changes to a new state S_{t+1} in time step $t+1$. At the end of the time step t for making a decision, a reward R_t is given to the agent, according to this reward the agent is guided to achieve his goal. The sequential process $S_t, A_t, R_t, S_{t+1}, A_{t+1}, \dots$ is the result of the agent's interaction with the intersection over time. Fig.1 [1].

Q-learning is a popular reinforcement learning algorithm [3]. It is a model-free reinforcement learning algorithm [6]. In other words, it does not build its own model of transition probability (P) and reward (R) matrices of the environment, but it directly evaluates the usefulness of taking an action A_t in a state S_t to calculate the value of a function called Q for

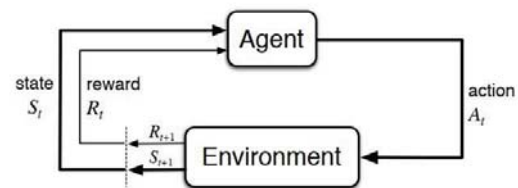


Fig. 1 The agent-environment interaction in RL problem

The pair (S_t, A_t) . The equation for calculating Q values for basic Q-learning is given in (1). This formula will be detailed in the following sections.

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha(r_{t+1} + \gamma \max_A Q(S_{t+1}, A_t) - Q(S_t, A_t)) \quad (1)$$

Tabular Q-learning cannot evolve into a large state space for complex traffic light control scenarios [3] [4] [5]. Samples with similar characteristics are treated as two completely different states, which will reduce the efficiency of using the samples during training. Function approximation $Q(s,a;\theta)$ (where θ is the hyperparameter of the approximator) of the Q-table was introduced to generalize a “function” from example to estimate the mapping. We use the methods with an approximation of the functions using a continuous representation of the states.

B. Deep Q Network

Deep Q-Network (DQN) is an algorithm using deep neural networks as a function approximation able to deal with reinforcement learning problems with a large action-state space. Divergence and instability are noted when using approximation functions to estimate Q values, and due to the strong correlation that exists between neural network inputs as well as frequent policy changes caused by updates in Q values.

Our system is a deep convolutional neural network (deep CNN) used to extract the characteristics of real-time traffic data (the position of vehicles, their speeds, the waiting time of each vehicle since it has integrated the intersection and the state of the traffic light) then make an optimal decision to control the traffic lights. In order to improve the agent's performance and to ensure its stability and the effectiveness of its learning, 2 mechanisms [7], replay memory [8] and target network, are employed during the training phase.

III. ADAPTATIVE SIGNAL CONTROL ALGORITHM BASED ON DQN

In this section, the intersection and its configuration parameters will be detailed, then we will express our problem of traffic light control as a Markovian decision process.

A. Intersection model

Our intersection model contains 4 traffic lights for each road Fig.2, vehicles on each road can turn left, right or go straight. Road traffic contains 8 entrances and exits through which vehicles can enter and leave the intersection. At every simulation step, a uniform process generates new vehicles, they are placed randomly on the routes entries. The entry time interval between two vehicles is calculated by a uniform distribution. The Table 1 shows the different configuration parameters.

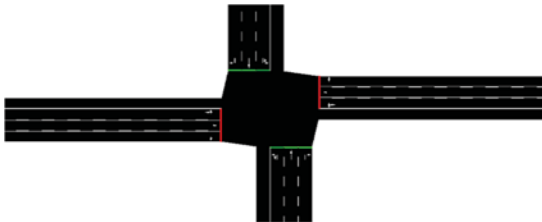


Fig. 2 Intersection model

TABLE 1 ROAD TRAFFIC NETWORK CONFIGURATION TABLE USED IN THE SIMULATION

Property	Value
Number of intersections	1
Number of routes	4
Average length of roads	500
Number of lines of each route	3
Maximum speed	70 km/h
Number of entry and exit	8
Average traffic flow	1000 veh/hour

B. State representation

The intersection status details which are provided to the agent, contain a vehicle position matrix, a vehicle speed matrix, and a vehicle waiting time matrix on each incoming route, as well as the status of the traffic lights. To generate these matrices on position, speed [7] [11] [12] [13] and waiting time of vehicle [9] [10], we will need the indices of the matrices. For this we must consider two parameters l (length of the segment) and c (length of the cell) for each route $i = 0, 1, 2, 3$ Fig. 3. For each route i there will be three matrices: vehicle position matrix P_i , vehicle speed matrix V_i and vehicle waiting time matrix W_i [9][10].

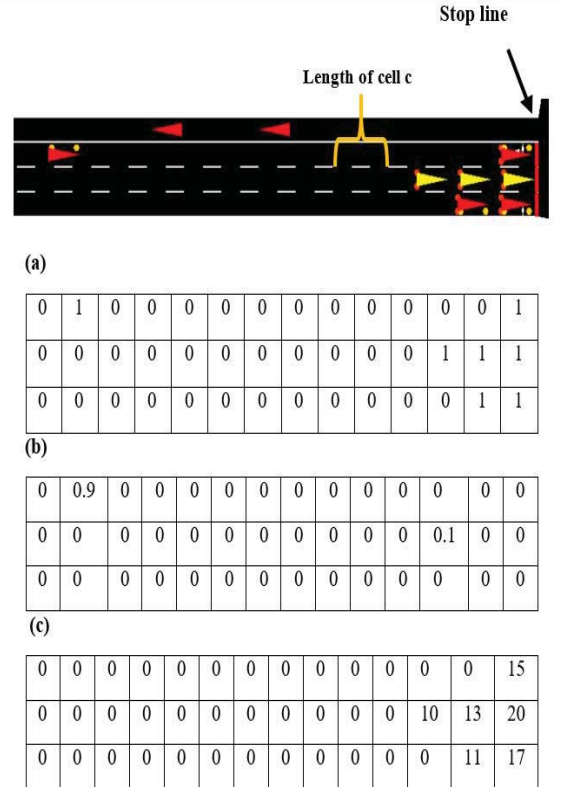


Fig. 3 snapshot of incoming road with (a) vehicle position matrix (b) vehicle speed matrix (c) vehicle wait times matrix

One-Hot Encoding is used for the position matrix for route **i**, the result is recorded at the corresponding input of the matrix **Pi**. The entry of the velocity matrix **Vi**, is the result of dividing the vehicle speed by the speed limit. The waiting time of each vehicle is recorded as an entry in the matrix **Wi**, this value is calculated from the moment the vehicle integrates the route **i** [9][10]. Another vector **L** of size 2 is used to illustrate the state of the traffic lights **L** = [1, 0] green lights the east-west roads and **L** = [0, 1] green lights the north-south roads [7]. At the beginning of the time step **t**, the agent will have a representation of the intersection as $S_t = (P, V, W, L) \in S$, where **S** is all the state space.

C. Action definition

The agent observes the state, and then he can choose one of two actions: **0** - green lights for the east-west roads and **1** - green lights for the north-south roads. If the taken action at time **t-1** does not change at time **t**, there will be no transition phase, otherwise, a transition phase will be added. The transition phase will run as follows: vehicles going directly will pass the yellow light, vehicles turning to the left will pass the green light, vehicles turning left will pass the yellow light, and vehicles turning left will go through the red light. All the yellow lights have a fixed duration of 6 seconds and the green lights of 10 seconds [7].

D. Reward definition

After the agent has chosen an action he will be rewarded, this will help him to understand the impact of his action on the environment. The reward guides the agent to choose better actions in the future. The reward has a crucial impact on the learning process. In general, the reward can be positive (reward) or negative (punishment). To define the agent's reward formula, we take two observations of the intersection at each time step (at the beginning and at the end). In each observation, the waiting times marked by the vehicles of the incoming lanes are noted. Our total reward R_t will be as follows.

$$R_t = r_1 - r_2 \quad (2)$$

At the time step **t**, r_1 is the sum of the waiting times of all the vehicles at the beginning of the green light interval. r_2 is the sum of the waiting times of all the vehicles at the end of the interval of green light [7]. If the waiting time r_2 decreases, the agent is rewarded; if the taken actions at time steps **t** and **t + 1** are not similar, then the environment will only be observed to record the two values r_1 and r_1 at the end of the transition phase explained in the section "action definition".

E. Deep Q network structure

As recall, the agent aims to reduce congestion, at the intersection in the long-term. We consider S_t the state of the intersection at the beginning of the time step **t**, after taking a series of decisions (actions) following a policy π . The agent receives a sequence of rewards $R_t, R_{t+1}, R_{t+2}, R_{t+3}, \dots$. Therefore, to reduce waiting times of vehicles at the intersection at time step **t**, the choice of action must maximize the immediate reward R_t as defined in (2). To achieve this goal, the agent is supposed to find an optimal policy for the choice of his actions, noted π^* defined in (4). This maximizing the cumulative future reward (Q values).

$$Q_\pi(s, a) = E[\sum_{k=0}^{\infty} \gamma^k R_{t+k} | S_t = s, A_t = a] \quad (3)$$

$$\pi^* = \operatorname{argmax}_\pi Q_\pi(s, a) \text{ for all } s \in S, a \in A \quad (4)$$

Where expectation is according to a policy of action π . Since the main subject of interest is the total reward, which is the expected reward that the agent receives through the sequence of all states, each reward is weighted by what is called the reduction factor $\gamma \in [0, 1]$. It is mathematically practical to discount rewards, as this avoids infinite returns in cyclical processes. Besides that the discount factor means that the more we are in the future, the less the rewards become important, because the future is often uncertain. We denote the optimal Q values under the policy π^* as $Q^*(s, a) = Q_{\pi^*}(s, a)$.

In this article, a deep convolutional neural network is used to extract useful traffic characteristics and thus finds the optimal policy for controlling traffic signals π^* . Finding an optimal policy π^* also means finding an action **a** that optimizes the Q value for the state **s** of the intersection. Therefore, the agent must find optimal Q values $Q^*(s, a)$. Bellman's optimality equation gives us a recursive relationship for optimal Q values $Q^*(s, a)$ (5).

$$Q^*(s, a) = E[R_t + \gamma Q^*(S_{t+1}, A_{t+1}) | S_t = s, A_t = a] \quad (5)$$

To solve this equation we need to have a finite number of total states and full knowledge of the underlying system mode (state transition probabilities and corresponding rewards) which in reality is impossible. The intersection traffic space is huge due to its complexity; this is why the use of the deep convolutional neural network to approximate the optimal Q values. The network has $Q(s, a, \theta) \approx Q^*(s, a)$ as output, where θ are the learned characteristics of the traffic data.

The architecture of our deep convolutional neural network follows the approach of [7] and [12]. $S_t = (P, V, W, L)$ state representation, is given as input for the CNN network, the output is a vector of estimated Q values $Q(S_t, a; \theta)$ for all actions $a \in A$ given the observed state S_t .

In Fig.4 we show in detail the architecture of the network, which is composed of three sub-networks identical receiving different inputs (P, V, W). Each sub-network contains three convolution layers of respectively: 16 filters of 4x4 with stride 2, 32 filters of 2x2 with stride of 1 and 64 filters of 2x2 with stride of 1, all the three are applying an activation function (ReLU). Then in the 4th and 5th layers, we have fully connected layers both applying an activation function (ReLU) of respective sizes of 128 and 64 units, the outputs of

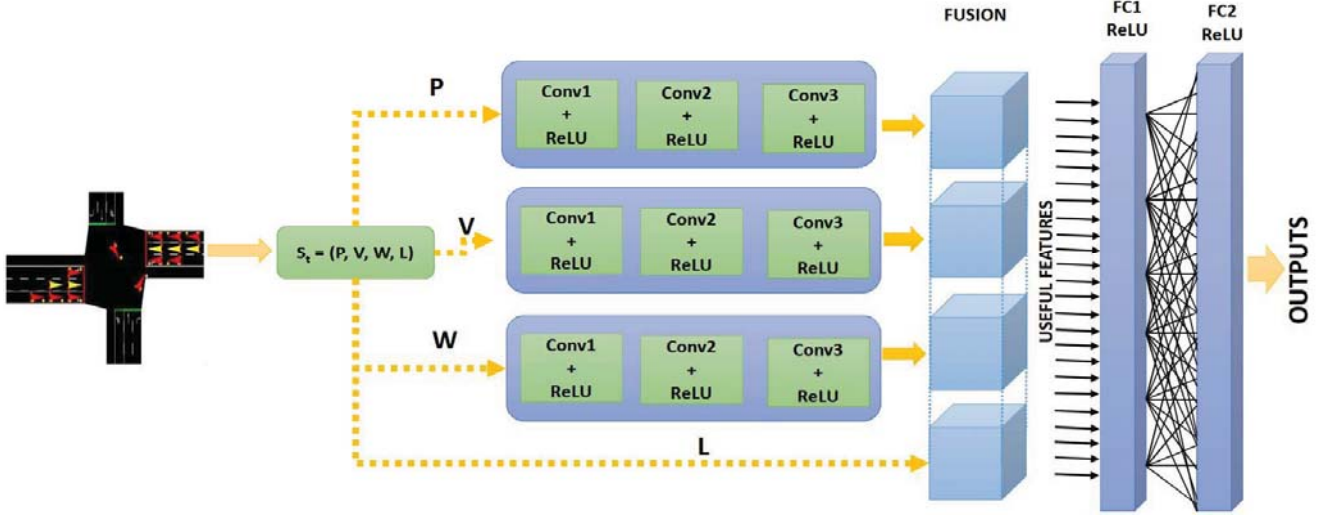


Fig. 4 Deep convolutional neural network architecture

the three previous sub-networks are flattened and the vector L of the last state lights will be concatenated and supplied to the 4th layer. The last layer is a fully connected linear layer of size 2.

F. Algorithm and training process

The algorithm in Fig.5 [7] summarizes the formation process of the deep convolutional network used. We use the memory replay technique [7] and [8], to improve the performance of the agent during the training phase. This technique submits to the agent a group of randomized samples called batch for its training, instead of immediately submitting the information that the agent collects during the simulation. The batch is extracted from a data structure called memory; each collected sample is stored in memory during the learning phase. A sample is formally defined as quadruple $\{S_t, A_t, R_{t+1}, S_{t+1}\}$. Where R_{t+1} is the received reward after performing the action A_t from state S_t , then the environment changes to the following state S_{t+1} . The transition that the environment undergoes from the state S_t to the state S_{t+1} , means that the observations extracted from this environment are correlated, which limits the agent's learning capacity. This is the reason for using the replay memory. During training, if the replay memory is full, it deletes the oldest experiences. To better estimate the optimal Q values Q^* , it is necessary to learn the parameters θ ($Q(s, a; \theta)$). The network has as input training data $X = \{(S_t, A_t): t \geq 1\}$ extracted from the replay memory. Since the optimal Q values are not known, they will be estimated as in [7] by $R_t + \gamma \max_{a'} Q(S_{t+1}, a'; \theta')$, where θ' are the parameters learned in the target network. This mechanism is used to solve the oscillation problem during the agent learning, and thus improves the algorithm stability. The target network has the same architecture as our deep convolutional neural network represented in Fig.4 and receives as input the state S_{t+1} of the sampled quadruple from the replay memory, the output of the target network is $Q(S_{t+1}, a'; \theta')$ and therefore the outputs are $y = \{R_t + \gamma \max_{a'} Q(S_{t+1}, a'; \theta'): t \geq 1\}$.

Once the outputs have been calculated, the deep convolutional neural network can be formed and the parameters θ are updated using the stochastic gradient descent RMSProp [14] with a mini-batch of size 32.

Following this method, during network formation, the agent randomly takes 32 quadruples ($\{S_t, A_t, R_{t+1}, S_{t+1}\}$) of the replay memory used to update the weights θ , because we use a target network whose weights are θ' , the agent also updates them but in such a so-called soft method [7] as follows:

$$\theta' = \beta \theta + (1 - \beta) \theta' \quad (6)$$

Where $\beta \ll 1$, it is the update rate.

The efficiency of traffic is better when an action with the highest value is selected at the current state; this action selection policy faces a frequent problem of reinforcement learning. In other words, this brings us to the exploration/exploitation dilemma; exploration, where we collect more information that could lead us to better decisions in the future; or exploitation, where we make the best decision based on current information. In this article, the ϵ -greedy exploration policy is chosen like in [12], following equation (7). It defines a probability ϵ for the current episode e of choosing an exploratory action, and consequently a probability $1 - \epsilon$ of choosing an exploiting action.

$$\epsilon_e = 1 - \frac{e}{Total_episodes} \quad (7)$$

By using this formula we guarantee that the agent does nothing but explore at the start of his training, a logical and obvious thing because he does not know his environment, he begins to use information acquired as he goes; whereas towards the end of his training, he was in exclusive exploitation.

Algorithm 1 Deep Q-learning with experience replay and target network

```

Initialize replay memory M
Initialize CNN network with random weights  $\theta$ 
Initialize target network with initial weights  $\theta' = \theta$ 
Initialize  $\epsilon, \beta, \gamma, \text{episodes}$ 
for episode = 1 to episodes do
    Start new time step
    while step  $\leq$  maxsteps do
        The agent observes  $S_t$ 
        The agent select  $A_t = \arg\max_a Q(S_t, a; \theta)$  with probability  $1 - \epsilon$ 
        and a random action with probability  $\epsilon$ 
        if  $A_t = A_{t-1}$  then
            No transition phase
        else
            Make a transition for traffic signals
        end if
        Implement selected action
        Increment simulation time
        if time step ends then
            Observe reward  $R_t$  and current state  $S_{t+1}$ 
            Store quadruple  $(S_t, A_t, R_t, S_{t+1})$  into M
            Randomly draw mini - batch of 32 samples from M
            Delete M[0] if the memory is full
            Form training data using data set X as inputs and y as targets
            Update weights of CNN network  $\theta$ 
            Update softly weights of target network  $\theta'$  according to equation (6)
        end if
    end while
end for

```

Fig. 5 Training algorithm

IV. SIMULATION RESULTS

A. Simulation Settings

As mentioned at the beginning of this paper, SUMO [2] is used to generate simulations of urban traffic. SUMO provides an infrastructure editor, a simulator interface, and an Application Programming Interface (API). The user relies on these elements to implement the configurations, to personalize the functionality of the road infrastructure, and to extract data during the traffic simulation.

The test of the method was carried out in the road traffic cited in Fig.2 at section II, it contains an intersection in a square distribution. The configuration of the road traffic network used in the simulation is presented in Table 1 at section II. The intersection contains three lines per road, with an average length equal to 700m. The maximum speed on each road is 70 km / hour. The vehicle flow is on average equal to 1000 vehicles per hour with a uniform distribution. To simulate heterogeneous traffic demands, the choice of vehicles for their entry lanes is done randomly. We have defined an asymmetry at roads, where the horizontal roads will be very busy while the vertical roads will be less used, and increasing the frequency for the roads deviating to the left compared to it deviating to the right. More precisely, $pH = 1/7$, $pV = 1/11$, $pAR = 1/30$, $pAL = 1/25$. The rules for activating the traffic lights used in this paper are detailed in the section II.

The artificial neural network was implemented using Keras [15]. For additional optimized functionality, we use NumPy and SciPy [16] libraries. Regarding agent training parameters, the training spans several episodes. The total number of episodes is 2000. The ϵ -greedy parameter is initially set to $\epsilon = 1$, the reduction factor $\gamma = 0.95$, the update rate of the weights of the target network $\beta = 0.001$, the

learning rate of RMSProp at 0.0002 and 200 episodes as size of the replay memory. By default, SUMO offers a time-frequency of 1 second per step, each episode lasts approximately 2 hours that represents a typical traffic load during “rush hour” that gives 7000 steps as the total number of steps per episode. The entire training takes about 35 hours on a laptop.

During the simulations, in a file, we record for all the episodes the sum of the waiting times of all the vehicles at the intersection at every second. The efficiency of our method with a network architecture Fig.4 and the inputs (P, V, W, L) explained in section III, is compared to the fixed configuration of traffic lights and to the approach of [7]. We used the approach of [7] without a target network mechanism. The performance of the two methods is presented in Fig. 6. Both approaches have shown a marked improvement over the fixed configuration of the signal lights. Using our agent, the obtained average of waiting times for all episodes was around 180k seconds while for the agent of [7] it was 230k seconds. It proves that a deeper architecture, as well as the two mechanisms of replay memory and the target network, have brought a great advantage to the problem of reducing traffic congestion.

B. Simulation results

As the traffic is asymmetrical, because the East / West traffic is greater than North / South one, the static configuration of the lights did not show good performance in terms of congestion reduction, because the execution of our simulations using static lights resulted in a total waiting time of approximately 338k seconds. Using our agent, the best result we could get was around 95k seconds, while the agent of approach [7] got 154k seconds. Our scores showed an improvement of around 72% compared to the fixed configuration. The average waiting time is around 180k seconds, proving that Q-Learning is an effective alternative for traffic light control.

The quantitative descriptions that the agent receives as a representation of the state for the environment are very important. The aim is to get a more complete description of the traffic situation. The states are more complicated but ensures a significant gain in terms of agent performance.

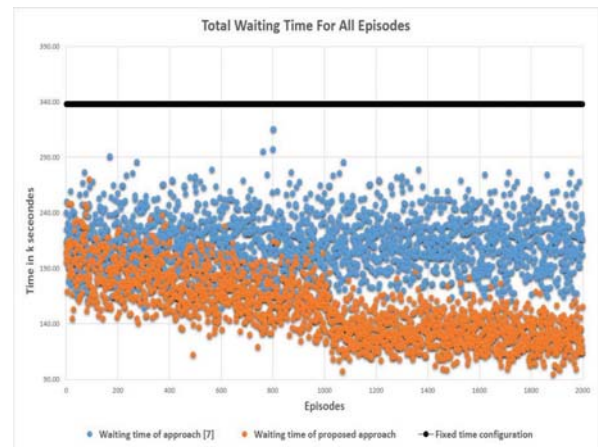


Fig. 6 Comprehensive view of improving waiting time

In our case, the introduction of an additional matrix of waiting times in the representation of the state made us get more performance; by comparing the obtained results from the proposed approach and those by applying the approach [7] at our intersection as showed in Fig.6.

As a real application, there are of course complications to be resolved, such as the lack of position, speed, and other details that allow the generation of state matrices. With the exception of these restrictions and assuming that intersections with appropriate detection technology allow vehicle detection, Q-learning has proven to be a viable alternative to traditional traffic control policies to reduce congestion on the world roads.

V. Conclusion

This work showed a credible exploration of the robustness of reinforcement learning in the adaptation and management of traffic lights in order to reduce traffic congestion. We used a realistic traffic simulator to provide an environment where the training and assessment of the agent can go well. Our algorithm was able to improve significantly the total waiting time of all vehicles at the intersection, using a deeper architecture for the neural network and adding more details to the state representation of the intersection by comparison with the method of [7]. It also improved the agent's learning stability and ensured convergence towards an optimal policy for choosing actions thanks to the two mechanisms of replay memory and the target network.

REFERENCES

- [1] Richard Sutton, A. G. Barto et al., "Introduction to reinforcement learning," MIT press Cambridge, vol. 135 (1998)
- [2] Behrisch, Michael and Bieker, Laura and Erdmann, Jakob and Krajzewicz Daniel, "Sumo – simulation of urban mobility: An overview," in SIMUL 2011, S. . U. of Oslo Aida Omerovic, R. I. R. T. P. D. A. Simoni, and R. I. R. T. P. G. Bobashev, Eds. ThinkMind, [Online]. Available: <https://elib.dlr.de/71460/> (2011)
- [3] Chris Watkins and Peter Dayan, "Q-learning," Machine learning, vol. 8, no. 3-4, pp. 279–292 (1992)
- [4] Chaimae El Hatri, Jaouad Boumhidi: Traffic management model for vehicle re-routing and traffic light control based on Multi-Objective Particle Swarm Optimization. *Intell. Decis. Technol.* 11(2): 199-208 (2017)
- [5] Mohammed Tahifa, Jaouad Boumhidi, Ali Yahyaouy: Multi-agent reinforcement learning-based approach for controlling signals through adaptation. *IJAACS* 11(2): 129-143 (2018)
- [6] Christopher John Cornish Hellaby Watkins, "Learning from delayed rewards," Ph.D. dissertation, King's College, Cambridge (1989)
- [7] Gao, Juntao & Shen, Yulong & Liu, Jia & Ito, Minoru & Shiratori, Norio, "Adaptive traffic signal control: Deep reinforcement learning algorithm with experience replay and target network," *arXiv preprint arXiv: 1705.02755* (2017)
- [8] Long-Ji Lin, "Reinforcement learning for robots using neural networks," Ph.D. thesis, Carnegie Mellon University (1993)
- [9] Hua Wei, Guanjie Zheng, Vikash Gayah, and Zhenhui Li. 2019. A Survey on Traffic Signal Control Methods. 1, 30 pages, 1 (2019)
- [10] Hua Wei, Guanjie Zheng, Vikash Gayah, and Zhenhui Li. 2020. A Survey on Traffic Signal Control Methods. 1, 32 pages, 1 (2020)
- [11] Wade Genders and Saiedeh. Razavi, "Evaluating reinforcement learning state representations for adaptive traffic signal control", *Procedia Computer Science*. doi:10.1016/j.procs.2018.04.008. (2018)
- [12] Wade Genders and Saiedeh. Razavi, "Using a deep reinforcement learning agent for traffic signal control," [Online]. Available: <https://arxiv.org/abs/1611.01142> (2016)
- [13] Shingate, Kranti, Komal Jagdale and Yohann Dias. "Adaptive Traffic Control System using Reinforcement Learning," *International journal of engineering research and technology* 9 (2020)
- [14] Tijmen Tieleman and Geoffrey Hinton, "Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude," COURSE: Neural networks for machine learning, vol. 4, no. 2, (2012)
- [15] François Chollet, "Keras," <https://github.com/fchollet/keras> [Accessed January.30, 2020]
- [16] Eric Jones, Travis Oliphant, Pearu Peterson , "SciPy: Open source scientific tools for Python," [Online]. Available: <http://www.scipy.org> (2001)